MINI PROJECT REPORT

ATTENDANCE MANAGEMENT SYSTEM

Student Name: Deepak R Chalmard

USN / Roll No.:1RUA25BCA0028

Course / Class: BCA (hons) / 'A'

Subject: Database management system

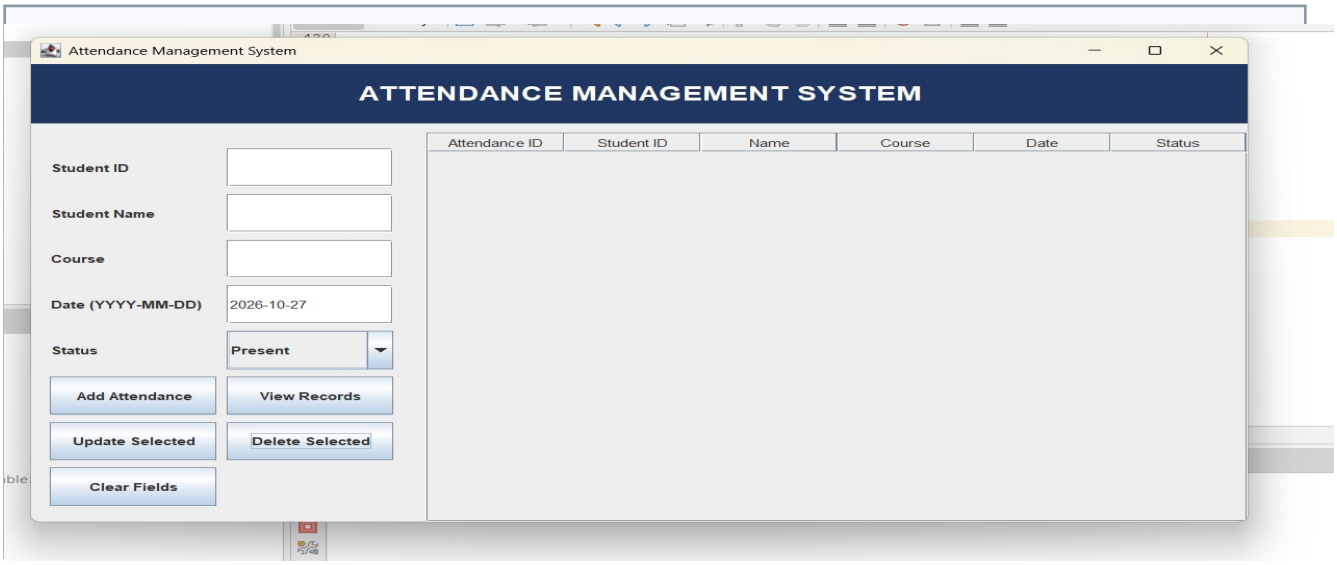
1. Project Objective

The objective of this project is to develop a simple desktop application that records student attendance and stores it in a MySQL database. The user can add attendance, view saved records, update a selected record, and delete a record through a Java Swing interface.

2. Tools and Technologies

Programming language: Java **Interface:** Java Swing **Database:** MySQL **Connectivity:** JDBC **IDE:** NetBeans **Database tool:** MySQL Workbench / MySQL command line.

3. Application Interface



Caption: The interface provides fields for student ID, student name, course, attendance date, and status, with buttons to add, view, update, delete, and clear records.

PAGE 2 — DATABASE DESIGN

4. Database and Table Creation

Database name: **attendance_db**. It contains two related tables: **students** stores student details and **attendance** stores attendance dates and statuses. The student_id foreign key links an attendance record to a student. A unique constraint prevents duplicate attendance for the same student on the same date.

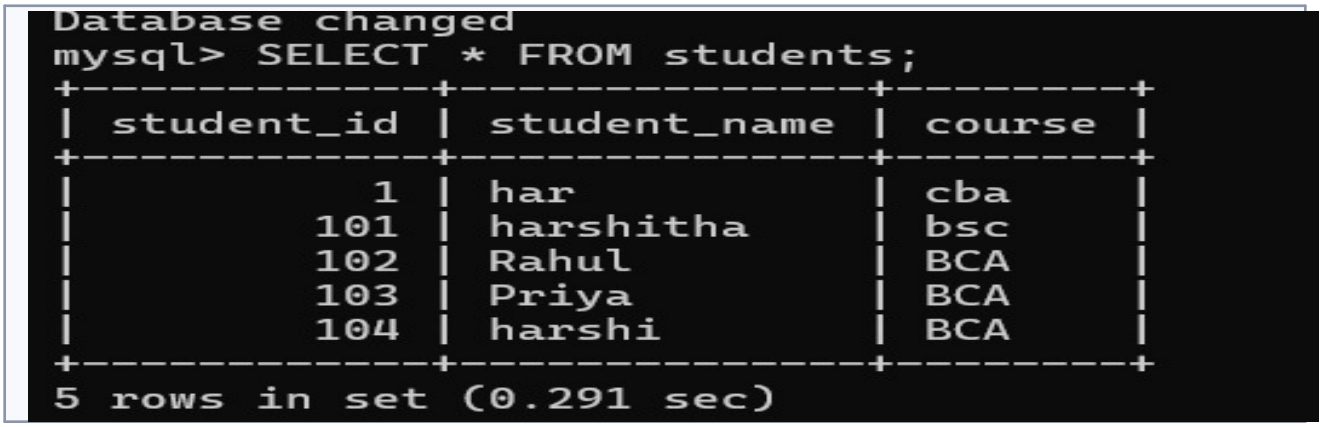
```
CREATE DATABASE IF NOT EXISTS attendance_db;
USE attendance_db;

CREATE TABLE IF NOT EXISTS students (
student_id INT PRIMARY KEY,
student_name VARCHAR(100) NOT NULL,
course VARCHAR(50) NOT NULL
);

CREATE TABLE IF NOT EXISTS attendance (
attendance_id INT AUTO_INCREMENT PRIMARY KEY,
student_id INT NOT NULL,
attendance_date DATE NOT NULL,
status ENUM('Present','Absent') NOT NULL,
FOREIGN KEY (student_id) REFERENCES students(student_id),
UNIQUE (student_id, attendance_date)
);
```

5. Sample Student Data

Student ID	Student Name	Course
101	Ananya	BCA
102	Rahul	BCA
103	Priya	BCA

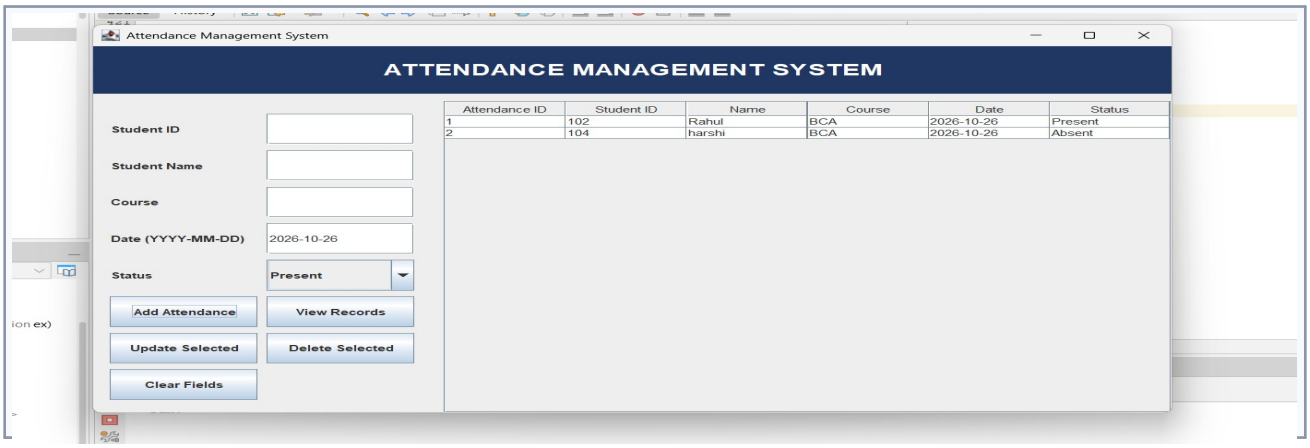


Caption: This screenshot should prove that the database and tables exist and that sample student records are stored in MySQL.

PAGE 3 — INSERT AND SELECT OPERATIONS

6. Data Inserted Through the Application

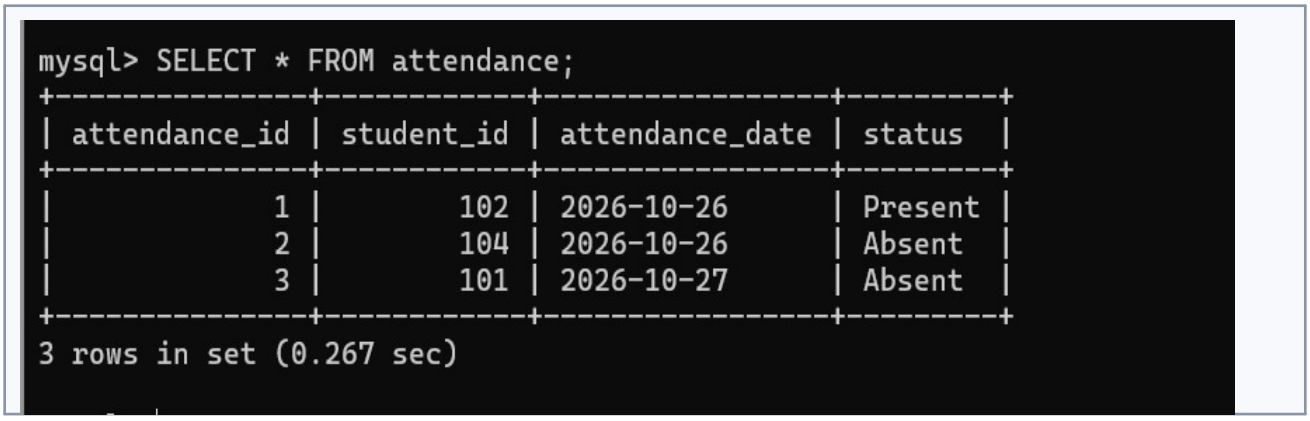
The user enters a student ID, name, course, date, and attendance status in the Java interface, then selects **Add Attendance**. The application uses JDBC and a prepared SQL statement to insert the student details and attendance record into MySQL. A success message should appear when the transaction completes.



7. Retrieving Saved Records

The **View Records** button runs a SELECT query joining the students and attendance tables. The result is displayed in the application's table. To verify persistence, run the following query in MySQL after inserting a record through the application:

```
USE attendance_db;
SELECT * FROM students;
SELECT * FROM attendance;
```

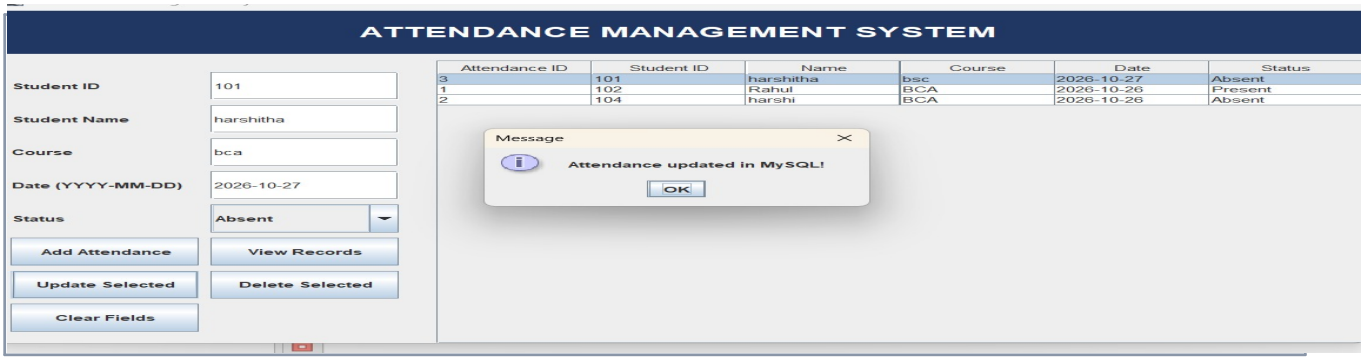


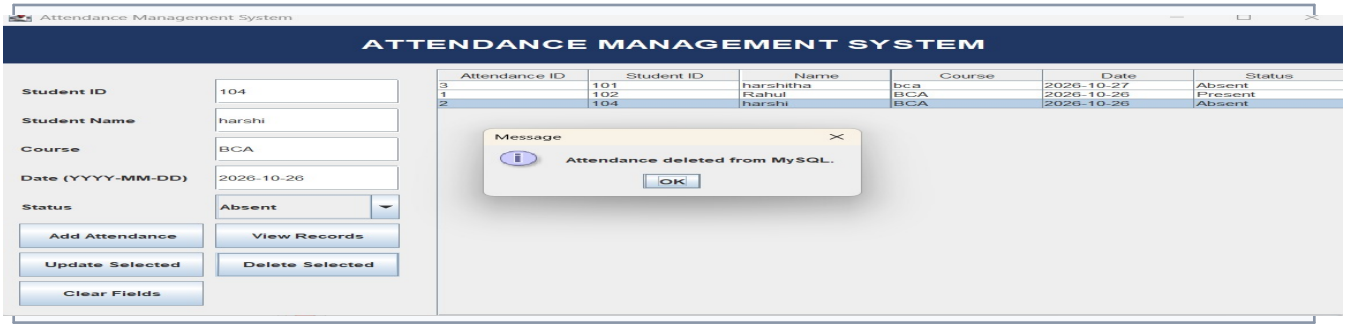
Caption: Include a real screenshot showing the newly added attendance record in the application or the corresponding MySQL query output.

PAGE 4 — UPDATE, DELETE AND TESTING

8. Updating and Deleting Records

Update: Select a row in the application, change the attendance date or status, and click **Update Selected**. The change should be reflected in MySQL.
Delete: Select a row, click **Delete Selected**, and confirm the action. The selected attendance record should be removed from MySQL.





9. Testing Summary

Test	Expected result	Actual result
Add attendance	Record saved in MySQL	_____
View records	Saved records displayed	_____
Update status/date	Changes stored in MySQL	_____
Delete record	Attendance row removed	_____
Invalid ID/date	Helpful validation message	_____

10. Conclusion

The Attendance Management System demonstrates Java–MySQL connectivity using JDBC and supports the basic database operations INSERT, SELECT, UPDATE, and DELETE. The application makes attendance records easier to manage and keeps saved data in the MySQL database.

Submission checklist: Java source files (AttendanceApp.java and DBConnection.java), SQL script (attendance.sql), this completed report with real screenshots, and a README containing setup and run instructions.